241801051

INDEX

NAME: DHANUSRI. R-S STD.: _____ SEC.: _____ ROLL NO.: 051 SUB.: Computer network

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	24/7/26	Network cmds used in Linux		
2.	6/8/26	Study of different types of network cable		
3.	23/7/26	Packet Tracer tool installation and user interface		
4.	6/8/26	Study of LAN using switch and Ethernet cable		
5.	6/8/26	Packet capture using wireshark		
6.	8/8/26	Error detection and correction using Hamming code		
7.	13/8/26	Flow control of Data Link Layer using Sliding window Protocal		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
9	24/8/26	Implementation of subnetting in cisco packet tracer		Done
10	24/8/26	Internet working using router		Done
10(b)	28/8/26	Wireless patter router, DHCP server and internet cloud		Done
11)	30/8/26	Routing at Network layer		Done
12	22/9/26	Implement echo client Server using TCP/UDP socket		Done
13	26/9/26	Ping program		Done
14	30/9/26	Network packet sniffer using Raw socket		Done
15	5/10/26	Types of Web logs using a Webalizer log analysis		Done completed

Ex: 01

24/07/26 NETWORK COMMANDS USED IN LINUX

AIM:

To study various network commands used is Linux and windows.

COMMANDS:

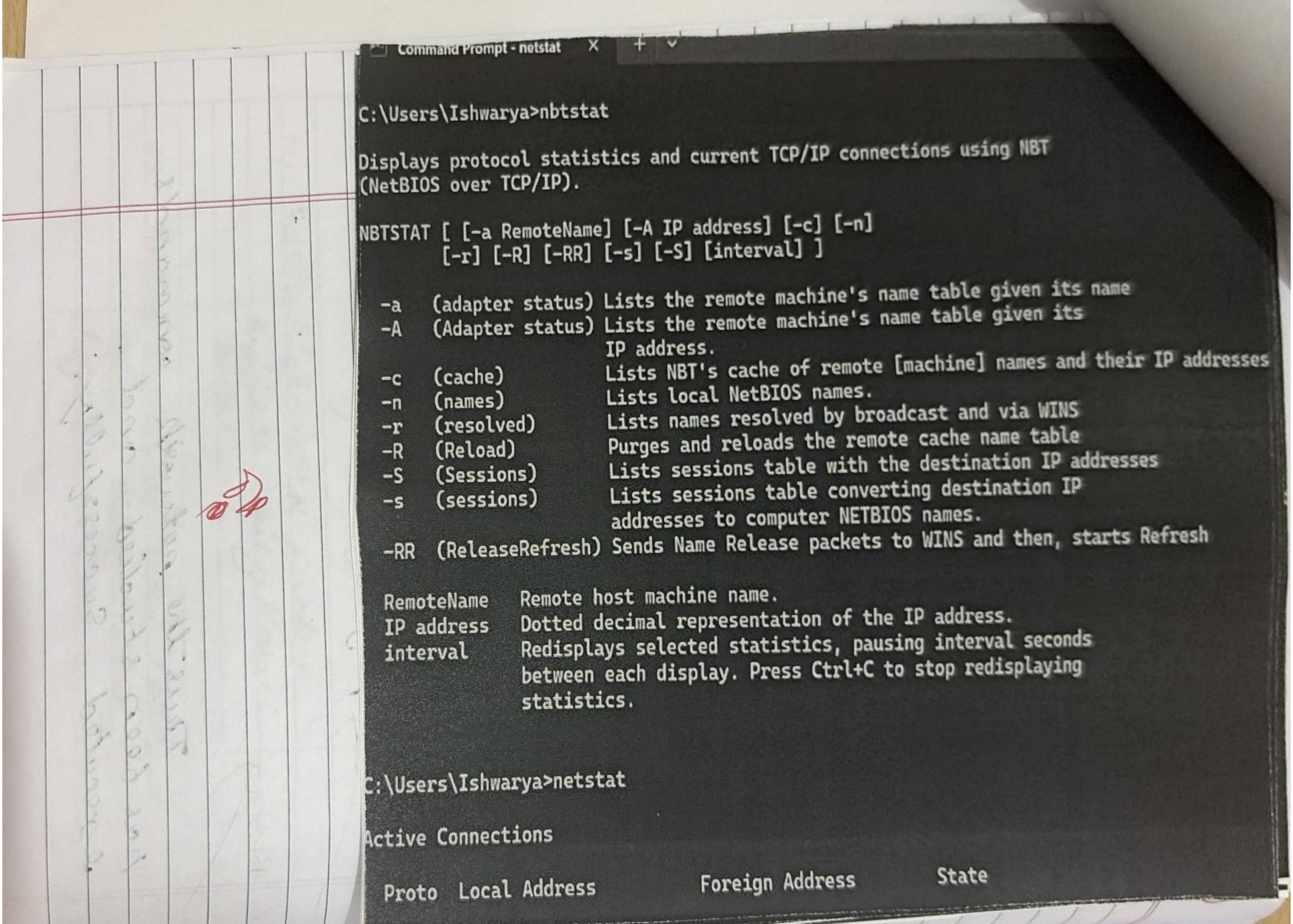
arp -a → Show IP address of computer,
hostname → Display name of computer
ipconfig → Display configuration of the PC.
nslookup → Display detail DNS.
pathping → Traces the root of destination address.
ping → Best way to test connectivity between nodes.
ping hostname
ping ipaddress
ping

RESULT:

Thus the network commands has been studied and executed successfully.

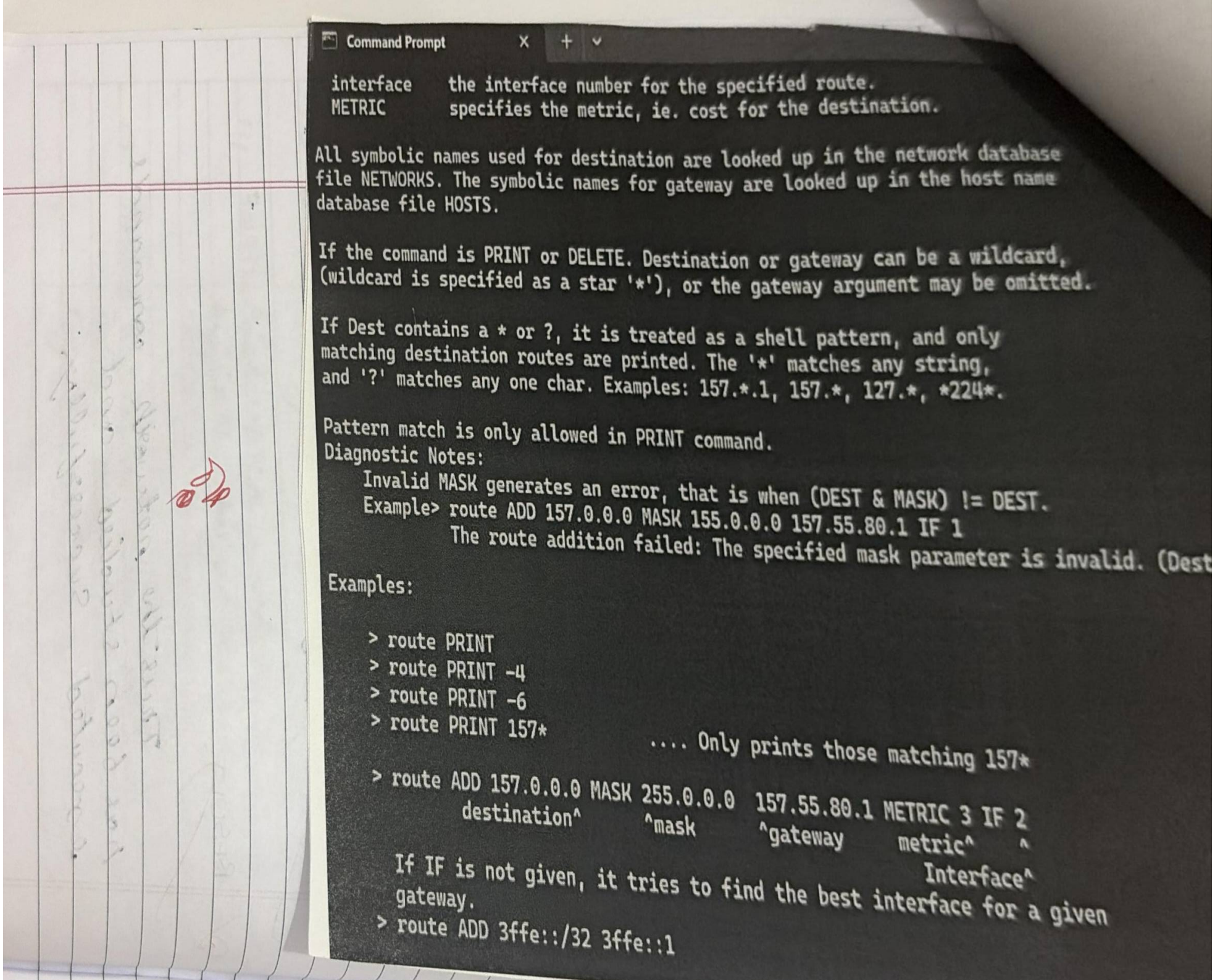
Command Prompt

```
Options:
    -g host-list      Loose source route along host-list.
    -h maximum_hops   Maximum number of hops to search for target.
    -i address        Use the specified source address.
    -n                Do not resolve addresses to hostnames.
    -p period         Wait period milliseconds between pings.
    -q num_queries    Number of queries per hop.
    -w timeout        Wait timeout milliseconds for each reply.
    -4                Force using IPv4.
    -6                Force using IPv6.

C:\Users\Ishwarya>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t                Ping the specified host until stopped.
                      To see statistics and continue - type Control-Break;
                      To stop - type Control-C.
    -a                Resolve addresses to hostnames.
    -n count          Number of echo requests to send.
    -l size           Send buffer size.
    -f                Set Don't Fragment flag in packet (IPv4-only).
    -i TTL            Time To Live.
    -v TOS            Type Of Service (IPv4-only. This setting has been deprecated
                      and has no effect on the type of service field in the IP
                      Header).
    -r count          Record route for count hops (IPv4-only).
    -s count          Timestamp for count hops (IPv4-only).
    -j host-list      Loose source route along host-list (IPv4-only).
```



Command Prompt - netstat

```
C:\Users\Ishwarya>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.


C:\Users\Ishwarya>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
```



Command Prompt

```
  interface     the interface number for the specified route.
  METRIC        specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Dest

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*                  .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
              destination^      ^mask      ^gateway     metric^    ^
                                                        Interface^
      If IF is not given, it tries to find the best interface for a given
      gateway.
    > route ADD 3ffe::/32 3ffe::1
```

EX:02 STUDY OF DIFFERENT TYPES OF NETWOR CABLES

6/8/26

AIM:

To study different types of network cables.

Understand different types of network cables:

1) Unshielded twisted pairs
2) Shielded twisted pairs
3) Coaxial cables
4) Fibre optic cable.

UPP)

Category 3
Category 5
Category 6C

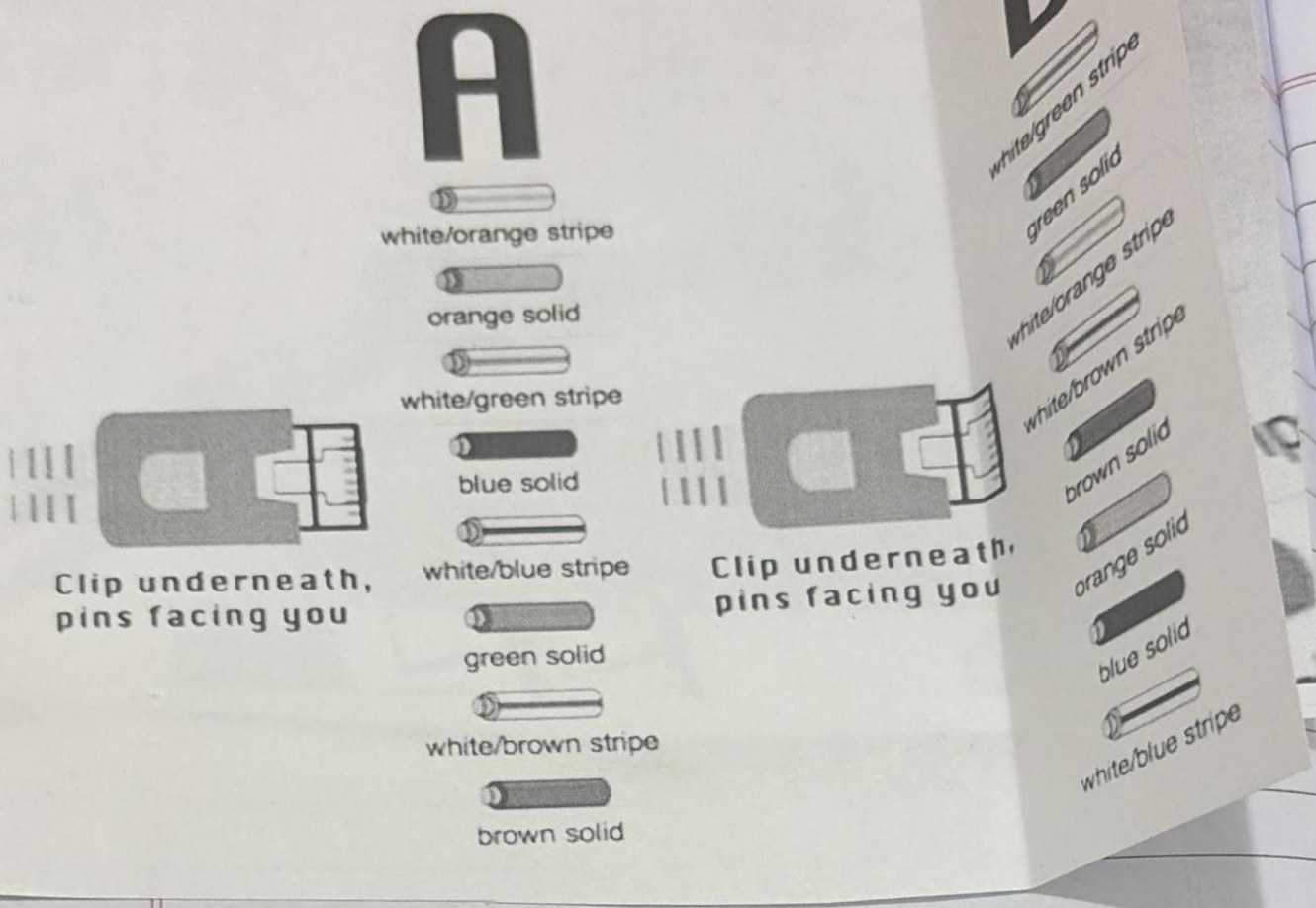
Application:

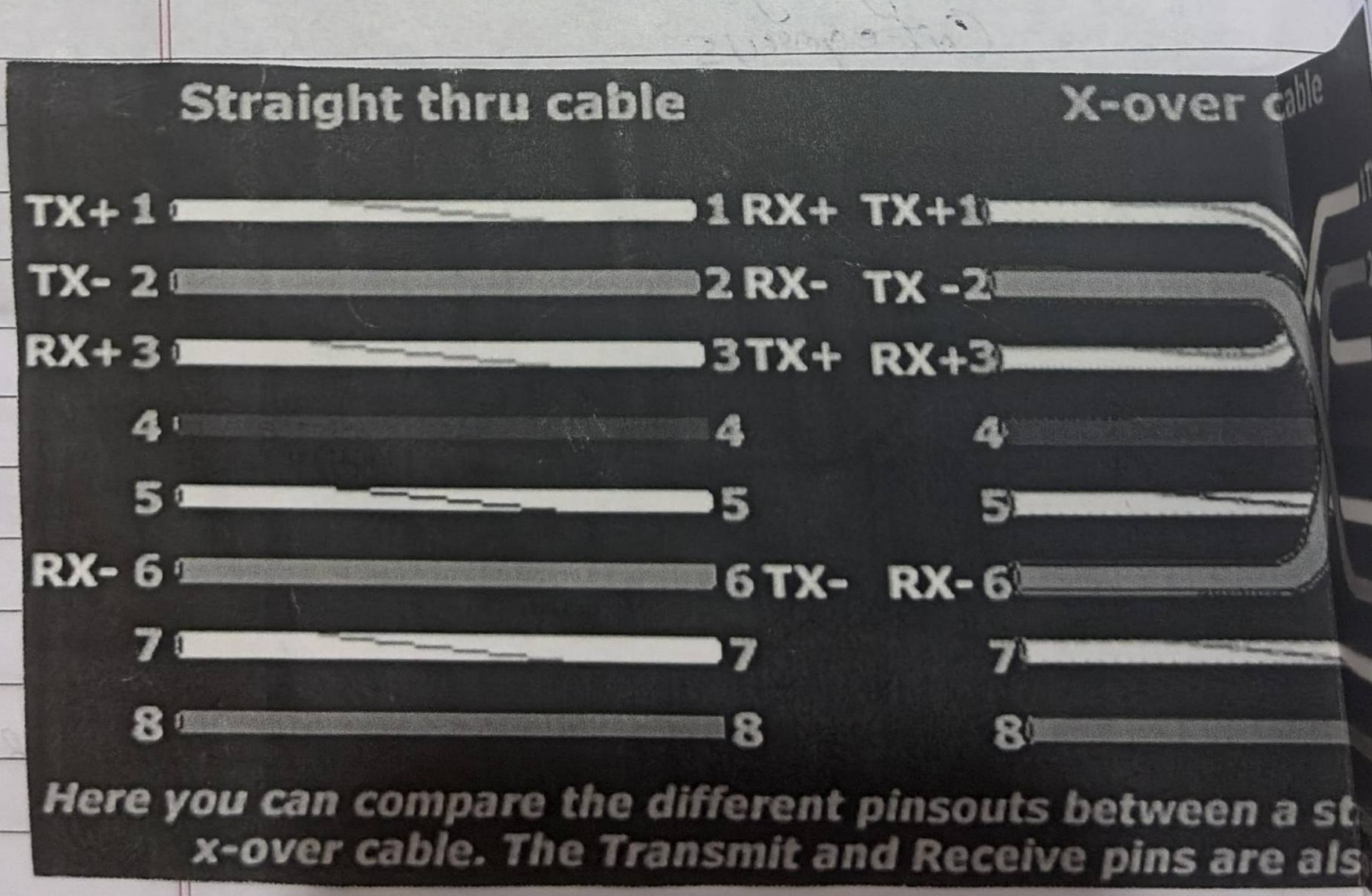
- 10-Base-T ethernet
- Fast Ethernet Gigabit

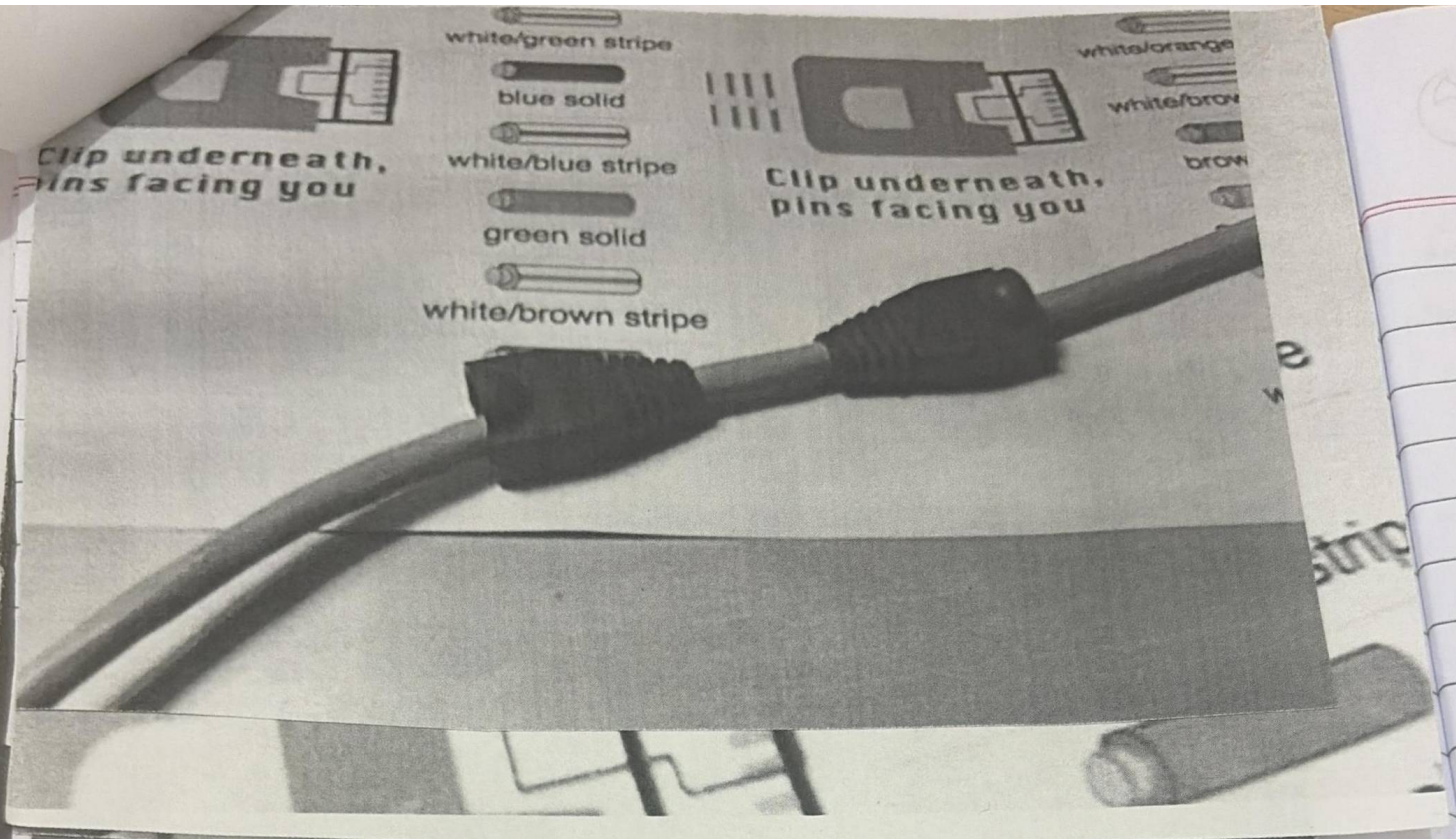
STP

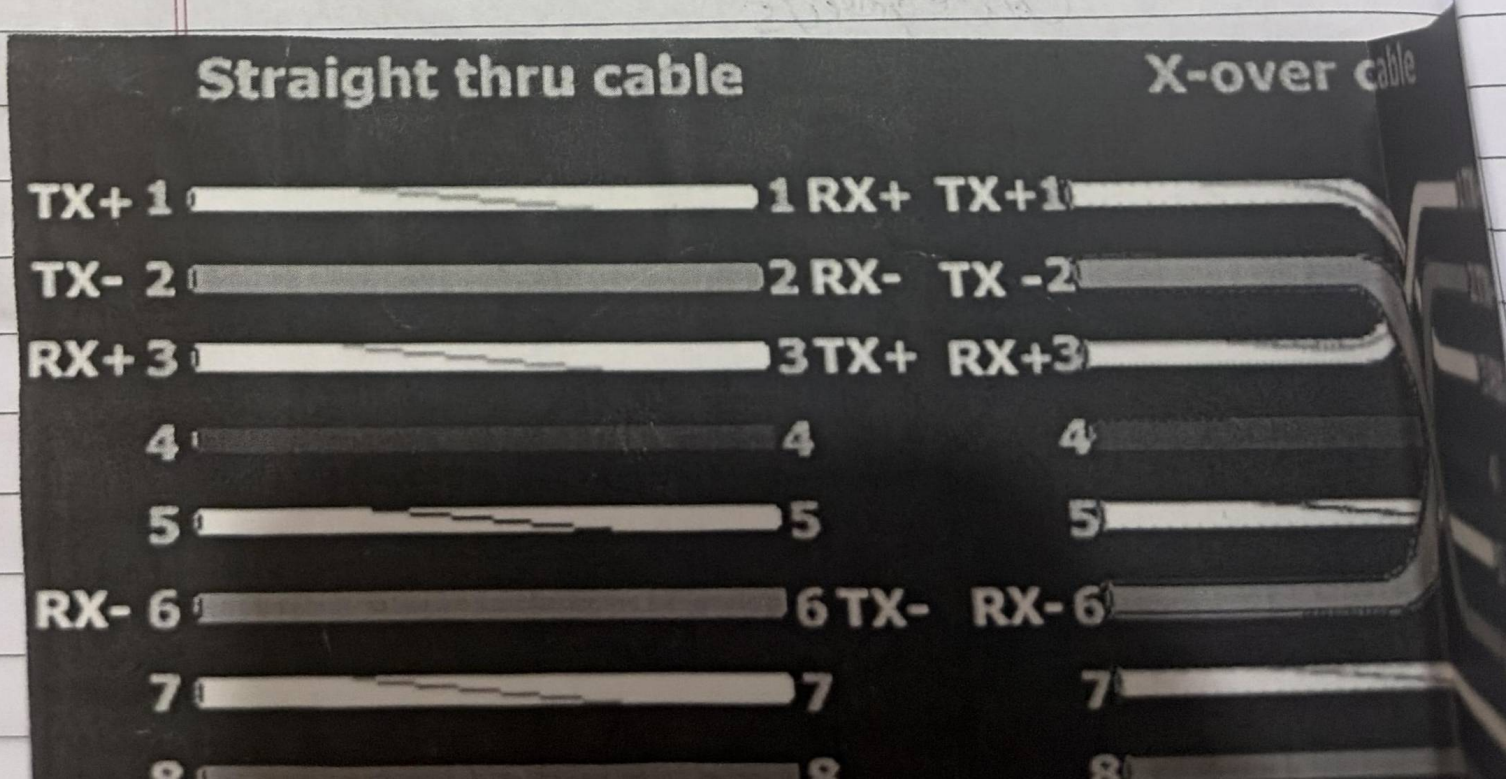
Category 6, 6a 10 Gbps Gigbit Ethernet. 10G Ethernet.

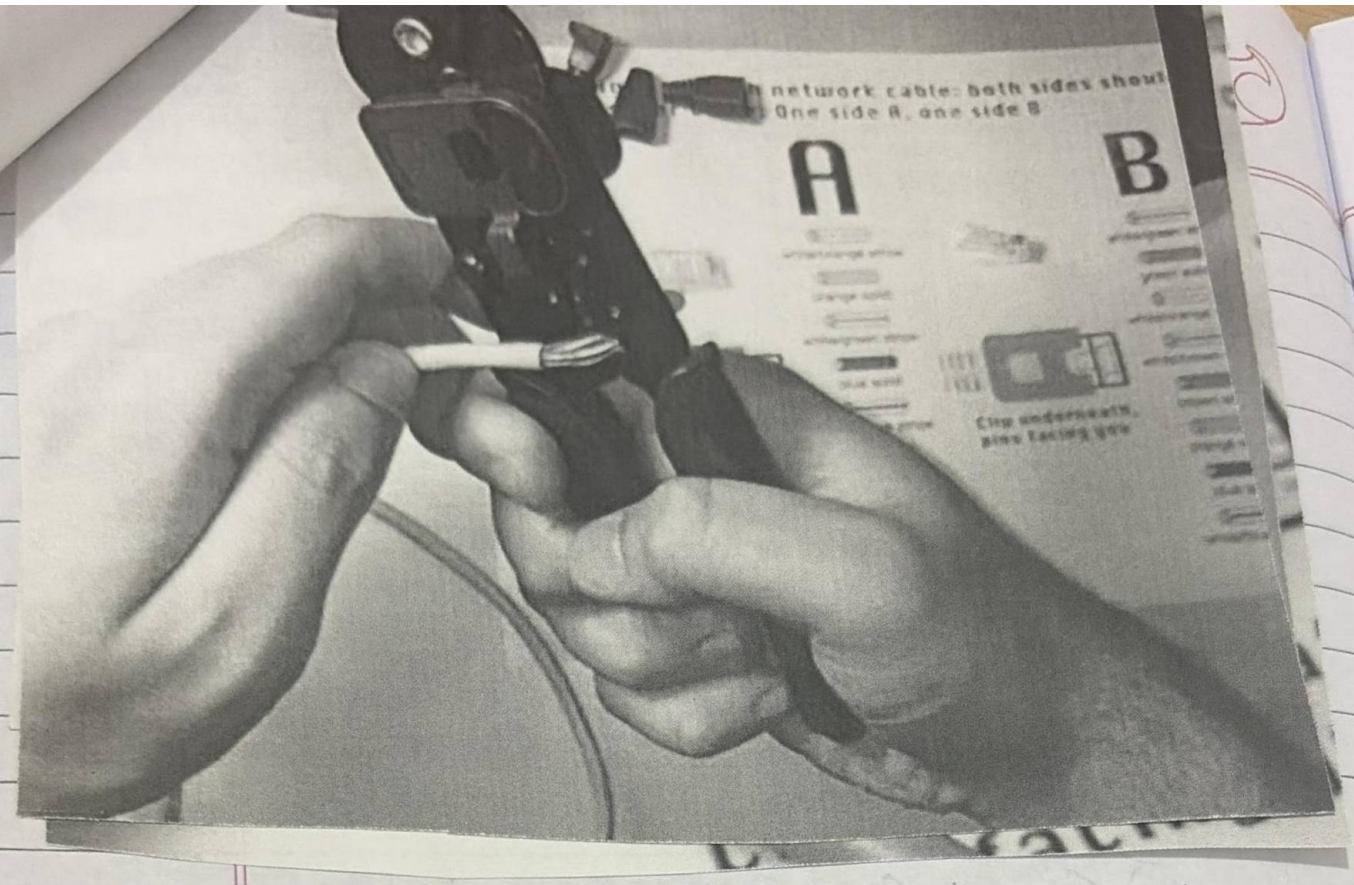
Straight through network cable: both s
Crossover cable: One side A, one side B






white/green stripe
blue solid
white/blue stripe
green solid
white/brown stripe
Clip underneath, pins facing you
Clip underneath, pins facing you


Straight thru cable
X-over cable
TX+ 1
1 RX+
TX+1
TX- 2
2 RX-
TX -2
RX+ 3
3 TX+
RX+3
4
4
4
5
5
5
RX- 6
6 TX-
RX- 6
7
7
7



Straight thru cable

X-over cable

TX+	1	1	RX+	TX+	1
TX-	2	2	RX-	TX-	2
RX+	3	3	TX+	RX+	3
	4	4			4
	5	5			5
RX-	6	6	TX-	RX-	6
	7	7			7
	8	8			8

Here you can compare the different pinsouts between a st

Advantage: Shielded, Faster than UTP

Coaxial cable:

RG-6 RG-59 RG-11 10-100 MBPS Speed of is signal 500m

Advantages: High bandwidth, Immune to interferance, Versatile

Fiber optic cable: High speed, High bandwidth, High security Long distance.

Single Node 100Gbps Maximum dis. of Bulk node is 100 meters.

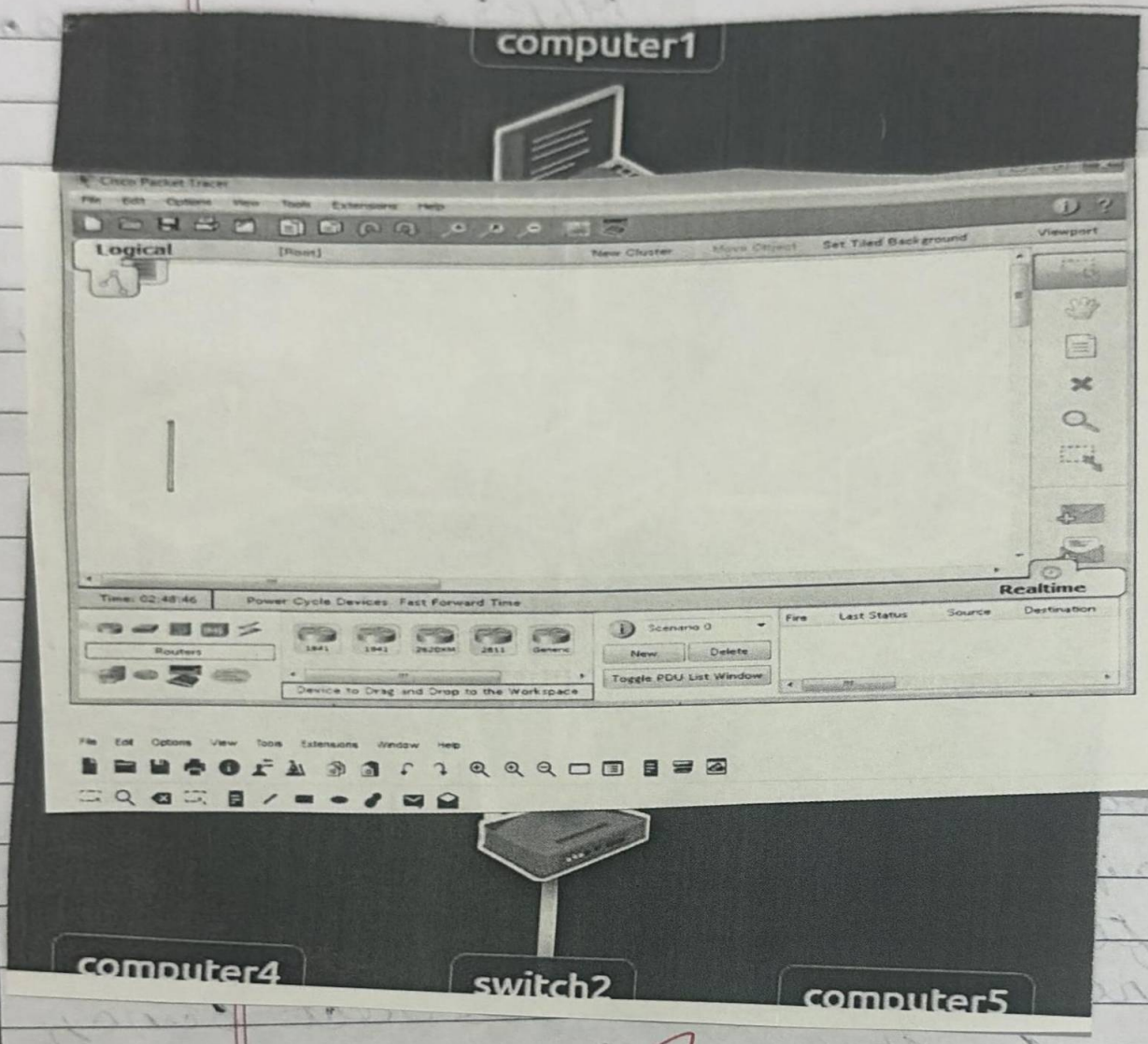
Disadvantage: Expenditure.

6) Make your own Ethernet.

- Start by threading shields onto table
- Strip aprox 1.5cm from both ends
- After will need to untangle wire.
- Once the order is correct bunch them together in line
- Lastly repeat the other end, to make a crossover cable.

RESULT:

The study of different types of cable has been studied completely.


computer1
Cisco Packet Tracer
Logical
[Root]
Realtime
Power Cycle Devices
Fast Forward Time
Device to Drag and Drop to the Workspace
computer4
switch2
computer5

EX:03
23/7/26

PACKET TRACER TOOL INSTALLATION AND USER INTERFACE

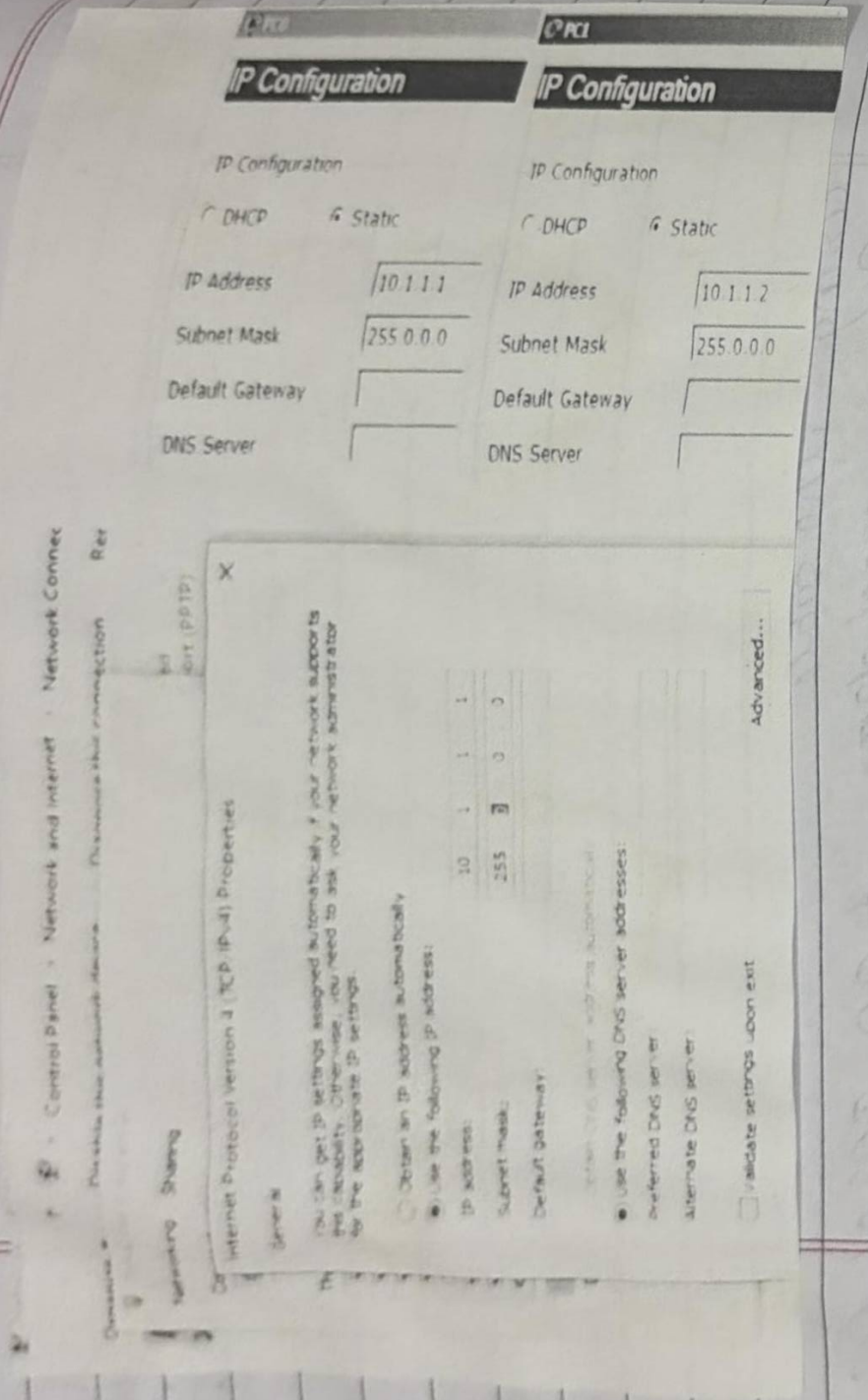
AIM: To study packet tracer tool: installation and interface connection.

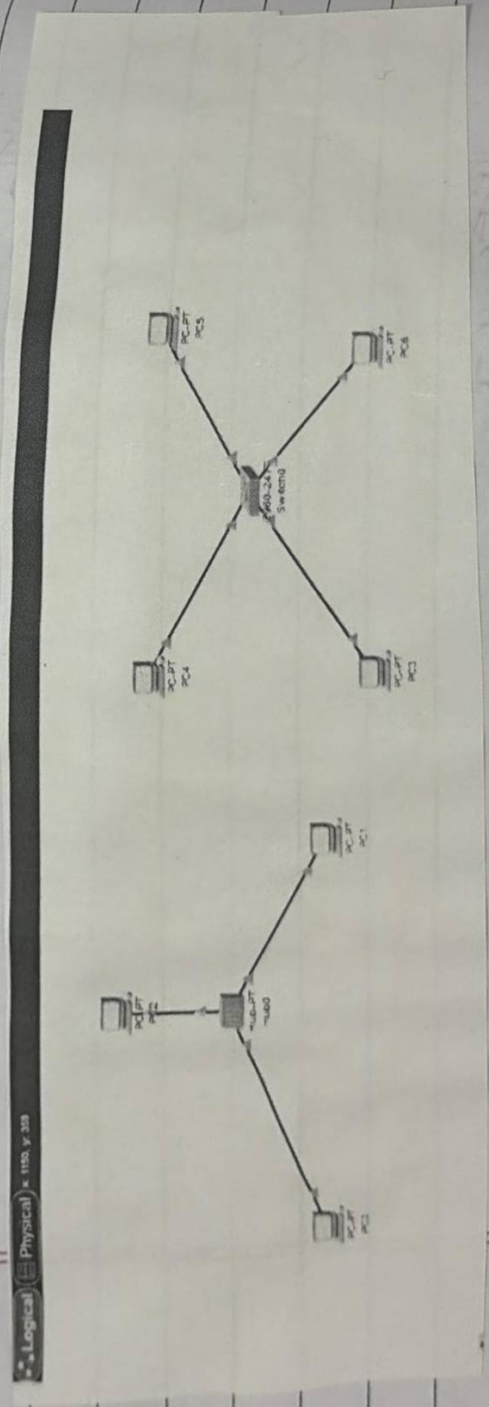
PROCEDURE:

1) From network components box click drag drop below components.
2) Click the connections:
   a) Drag select one of the PC and connect it to HUB using the cable
3) Drag and drop it on one of the PC and drag it on destination.
4) Observe the flow of POU from source PC.
5) Repeat step 3-5 for PC connected.
6) Observe how hub and switch are forwarding the POC and write your observation.

RESULT:

Thus the study of packet trace tool installation and user interface overview was successfully implemented


PC1
IP Configuration
IP Configuration
DHCP
Static
IP Address
10.1.1.1
Subnet Mask
255.0.0.0
Default Gateway
DNS Server
PC1
IP Configuration
IP Configuration
DHCP
Static
IP Address
10.1.1.2
Subnet Mask
255.0.0.0
Default Gateway
DNS Server
Control Panel
Network and Internet
Network Connec
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
Advanced...


Logical
Physical

Ex:4
6/8/2026

STUDY OF LAN USING A SWITCH AND ETHERNET CABLE

AIM:

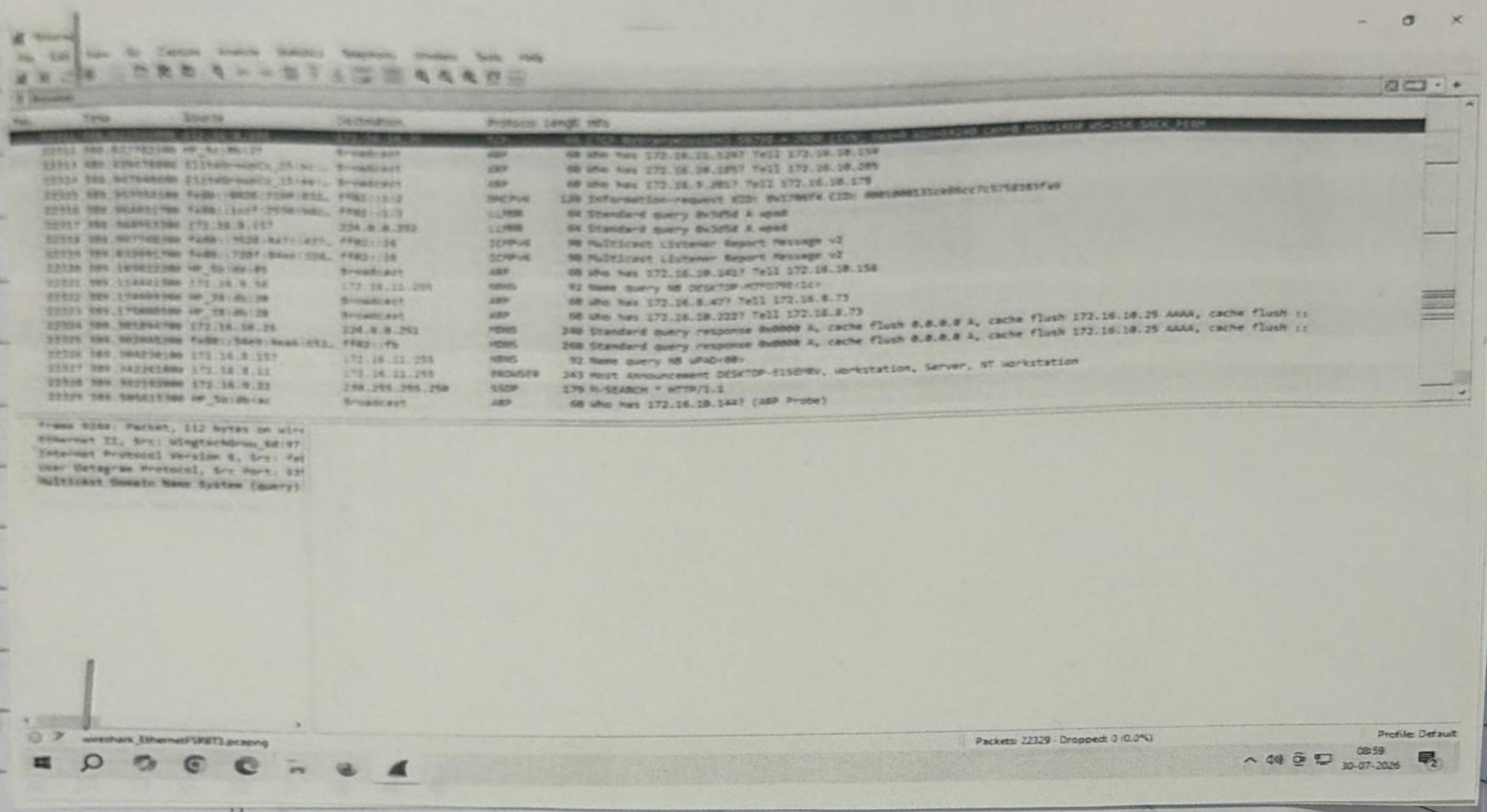
To setup and configure a local area network using a switch and ethernet cable and verify communication b/w connected computers.

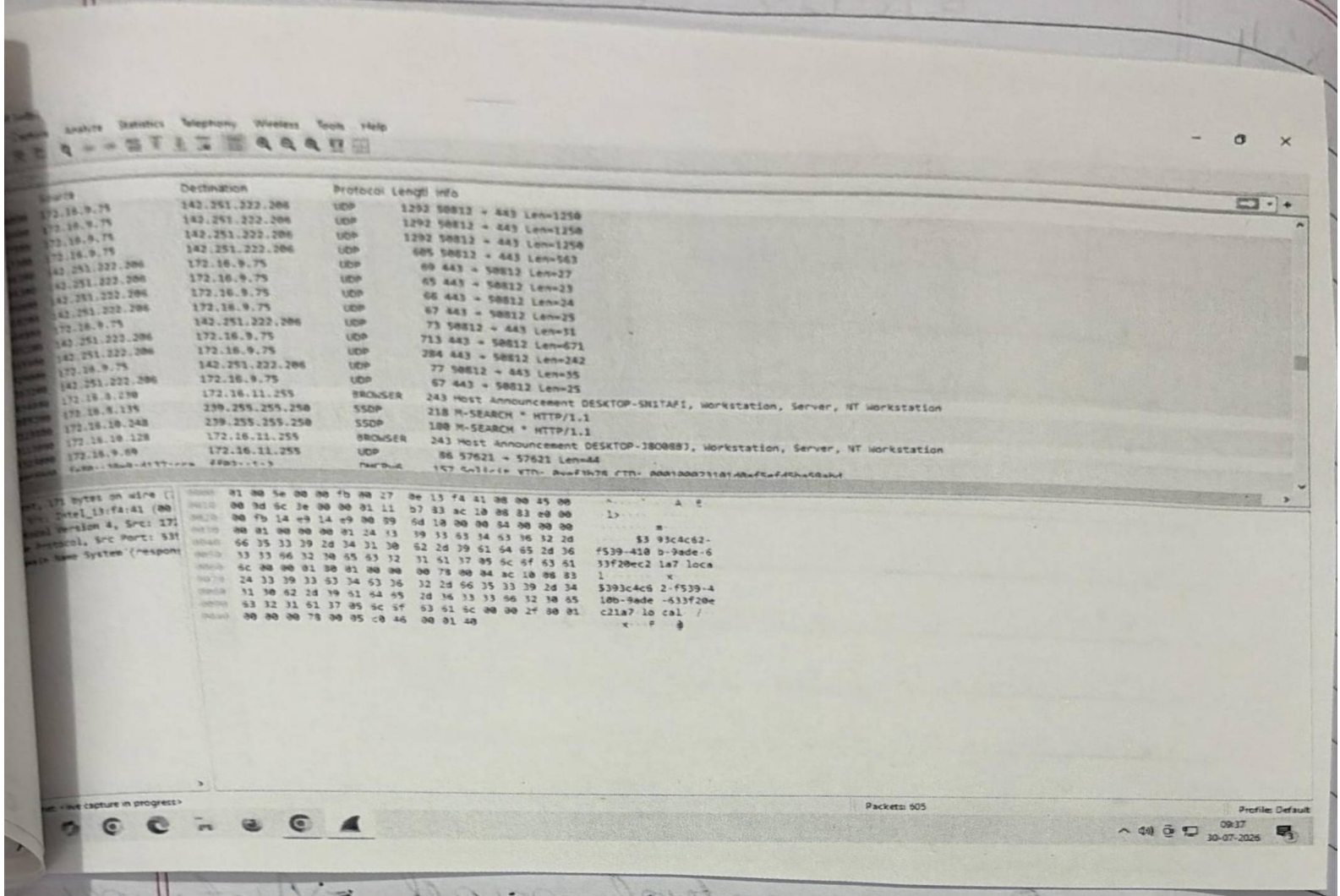
PROCEDURE:

- Plan the network by arranging 4 computers, one network switch and ethernet cable.
- Connect each computer to switch using ethernet cable
- Open control panel → Network → Internet → Network connection
- Right click on internet properties
TCP:
- Select configure the switch by connecting it to a computer, accessing it, web interface and assigning switch IPas 10.1.1.5 with 255.00.0

RESULT:

Thus the LAN was successfully setup and configured.


Time
Source
Destination
Protocol Length Info
ARP 60 Who has 172.16.11.126? Tell 172.16.10.150
ARP 60 Who has 172.16.10.105? Tell 172.16.10.205
ARP 60 Who has 172.16.9.205? Tell 172.16.10.179
DHCPv6 120 Information-request XID: 0x17869f4 CID:
LLMNR 84 Standard query 0x7d5d A wpad
LLMNR 84 Standard query 0x7d5d A wpad
ICMPv6 90 Multicast Listener Report Message v2
ICMPv6 90 Multicast Listener Report Message v2
ARP 60 Who has 172.16.10.156
NBNS 92 Name query NB DESKTOP-H7F07P8<1c>
ARP 60 Who has 172.16.8.73? Tell 172.16.8.77
ARP 60 Who has 172.16.10.222? Tell 172.16.8.73
MDNS 240 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush ::
MDNS 268 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush ::
NBNS 92 Name query NB WPAD<00>
BROWSER 243 Host Announcement DESKTOP-E15DMEV, Workstation, Server, NT Workstation
SSDP 179 M-SEARCH * HTTP/1.1
ARP 60 Who has 172.16.10.144? (ARP Probe)
Frame 9264: Packet, 112 bytes on wi
Ethernet II, Src: WingtechMobile_6d:97
Internet Protocol Version 4, Src: fe
User Datagram Protocol, Src Port: 13
Multicast Domain Name System (query)
wireshark_EthernetF1RBT3.pcapng
Packets: 22329 · Dropped: 0 (0.0%)
Profile: Default
08:59
30-07-2026


Analyze Statistics Telephony Wireless Tools Help
Destination Protocol Length Info
142.251.222.206 UDP 1293 50812 → 443 Len=1250
142.251.222.206 UDP 1292 50812 → 443 Len=1250
142.251.222.206 UDP 1292 50812 → 443 Len=1250
142.251.222.206 UDP 605 50812 → 443 Len=563
172.16.9.75 UDP 69 443 → 50812 Len=27
172.16.9.75 UDP 65 443 → 50812 Len=23
172.16.9.75 UDP 66 443 → 50812 Len=24
172.16.9.75 UDP 67 443 → 50812 Len=25
142.251.222.206 UDP 73 50812 → 443 Len=31
172.16.9.75 UDP 713 443 → 50812 Len=671
172.16.9.75 UDP 284 443 → 50812 Len=242
142.251.222.206 UDP 77 50812 → 443 Len=35
172.16.9.75 UDP 67 443 → 50812 Len=25
172.16.11.255 BROWSER 243 Host Announcement DESKTOP-SN1TAFI, Workstation, Server, NT Workstation
239.255.255.250 SSDP 218 M-SEARCH * HTTP/1.1
239.255.255.250 SSDP 180 M-SEARCH * HTTP/1.1
172.16.11.255 BROWSER 243 Host Announcement DESKTOP-JBO08BJ, Workstation, Server, NT Workstation
172.16.11.255 UDP 86 57621 → 57621 Len=44
Packets: 605
Profile: Default
09:37
30-07-2026

EX:5
6/8/26

PACKET CAPTURE USING WIRESHARK

AIM:

To capture filter inspect and analyse network packets using the wireshark packet analyzer.

ALGORITHM:

1) Open wireshark.
2) Select active network interface
3) Configure capture to stop after 100 packets.
4) Start packet capture
5) Apply display filters
6) Inspect packet details.
7) Save the packets

PROCEDURE:

- Launch wireshark and subset
- Go to capture → options set stop capture after 100 packets
- Inspect the filtered packets list, details and bytes
- Save the capture file

RESULT:

Successfully captured 100 network packets applied display filters and analysed details in wireshark.

EX: 6
8/8/2024

ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM:

To write a program to implement error detection and correction using hamming code.

ALGORITHM:

* Start the program
* Read the number of 4-digit data block.
* For each digit a bit of data block in perform hamming
  $P_1 = d_1 \times OR\ d_2 \times OR\ d_4$
  $P_2 = d_1 \times OR\ d_3 \times OR\ d_4$
  $P_3 = d_2 \times OR\ d_3 \times OR\ d_4$
* Simulate transmission by introducing a single bit error.
* Repeat step 3 to 8. for all n blocks in data.
* Stop the program.

PROGRAM:

```
h = [0]*7
h[2] = d[0]
h[4] = d[1]
h[5] = d[2]
h[6] = d[3]
h[0] = h[2] ^ h[4] ^ h[6]
```

```
h[3] = h[4] ^ h[5] h[6]
def main():
    n = int(input("Enter number: "))
    for b in range(1, n+1):
        d = list(map(int, input()))
    h = encode(d)
    if 1 <= pos <= 7:
        h[pos-1] ^= 1
        print(f"Transmitted
        cod with error:
        {to-str(4)}")

    else{
        print(f"Transmitted
        code: {to-str(n)}")

Print(f"Received code: {to-
        str(n)}")
detect-and-correct(h)
print(f"corrected code:
        {to-str(h)}")

rd = [h[2], h[4], h[5],
        h[6]]

Print(f"corrected data bits: {to-
        str(cord)}")
```

OUTPUT:

Enter number of 4bit blocks: 2

--Block 1--

Enter 4 data bits: 1011

Original hamming code: 1011

Enter bit position to introduce error: 3

Transmitted code: 0100111

Recivected code data bit: 1011

--Block 2--

Enter 4 data bits: 0110

Original data bits: 0110

Encoded hamming code: 1100110

Transmitted code: 1100100

Received code: 1100100

Corrected code: 1100110

Recovered data bits: 0110

```
if __name__ == "__main__":
    main()
```

RESULT:

The program to implement error detection and correction using Hamming code concept was successfully executed

OUTPUT:

Enter total no of frame to send : 8
Enter Sender's window Size : 4
Enter frame no to simulate
at cost : 5

Senders window (from 0 - frame)
Node A -> senders frame 0
Node A -> Sending frame 1
Node A -> sending frame 2
Node A -> sending frame 3
Node B -> Received frame 0
Node B -> received frame 1
Node B -> received frame 2
Node B -> received frame 3

Senders window (frame 4 - 7)

Node A -> sending frame 4
Node A -> sending frame 5
Node A -> sending frame 6
Node B -> frame 5 missing
Node A -> Timeout
Node A -> ACK - received sliding window
All 8 frames successfully
transmitted from Node
A to Node B.

EX: 07
13/8/2026

FLOW CONTROL OF DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

AIM:

To write a program to implement flow control at data link layer using sliding window layer protocol.

ALGORITHM:

1. Start the program.
2. Load total no. of frames to be sent (N).
3. Initialise base = 0.
4. Repeat while base < N
   a) Determine current sender window as range (base, base + N - 1)
   b) Send all the frames with sequence number.
   c) If the frame lies within frames first sent
   d) The receiving accepts each frame in order and sends a cumulative ack.
5) Repeat steps until all N frames transmitted
6) Stop

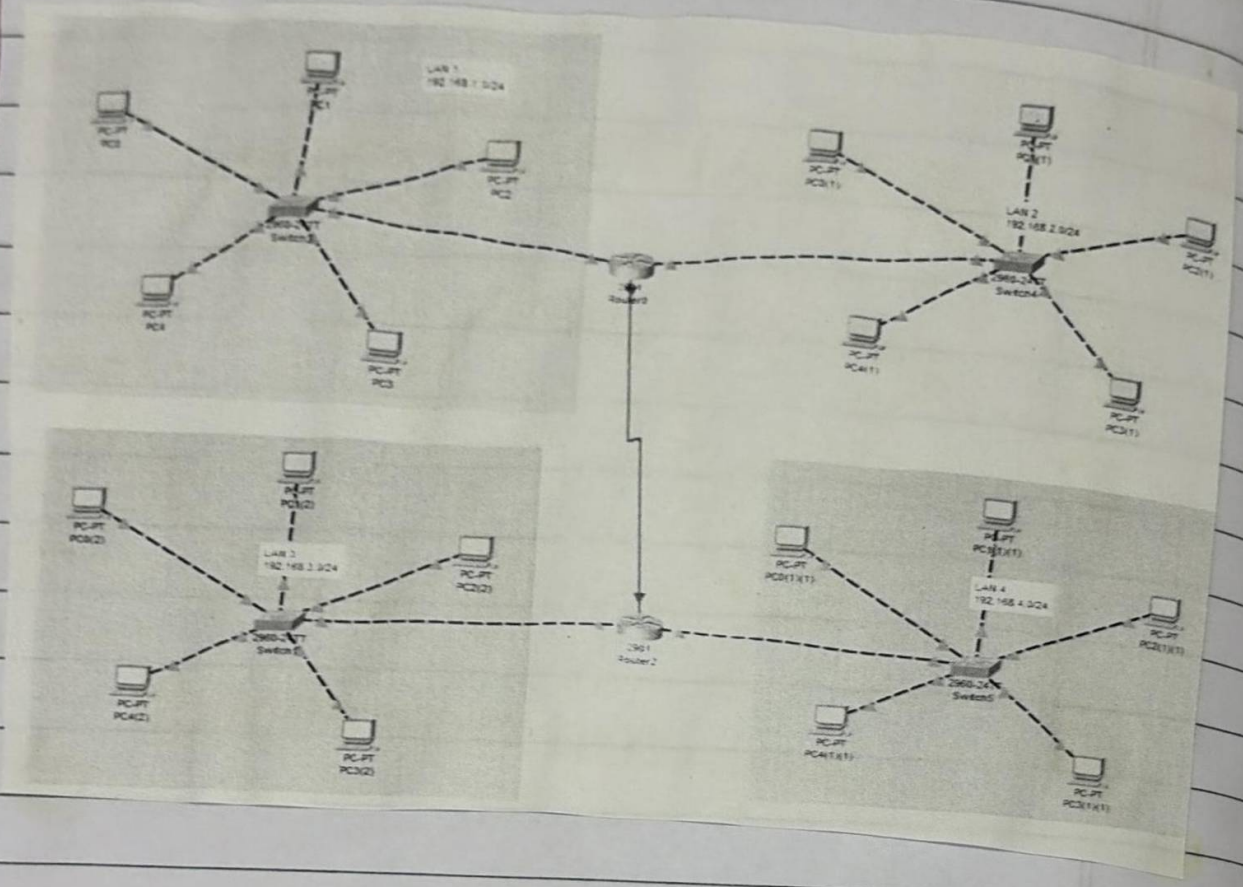
PROGRAM:

```
def simulate(total_frames, window_size):
    base = 0
    next_seq = 0
    while base < total_frames:
        window_end = min(base + window_size - 1, total_frames - 1)
        while next_seq < base + window_size and next_seq < total_frames:
            if next_seq == lost_frame:
                print(f"Frame {next_seq} is lost in transit.")
            next_seq += 1
        if lost_frame is not None and base <= lost_frame:
            next_seq = lost_frame
            lost_frame = None

def main():
    total_frames = int(input("Enter total frames at end: "))
    lost_frame = -1
    if lost_frame == -1:
        lost_frame = None

simulate(total_frames, window_size, lost_frame)
```

```
if __name__ == "__main__":
    main()
```

RESULT:

The program to implement flow control at the data link layer using sliding window protocol was successfully executed by simulating flow of frames.


LAN 1
192.168.1.0/24
PC-PT
PC0
PC1
PC2
PC3
PC4
Switch0
Router0
LAN 2
192.168.2.0/24
PC2(1)
PC3(1)
PC4(1)
Switch4
LAN 3
192.168.3.0/24
PC0(2)
PC2(2)
PC3(2)
PC4(2)
Switch3
Router2
LAN 4
192.168.4.0/24
PC2(1)(1)
PC3(1)(1)
PC4(1)(1)
Switch5

EX:09 IMPLEMENTATION OF
24/8/26 SUBNETING IN CISCO PACKET TRACER

AIM:

To implement classless IP subnetting using cisco packet tracer simulator and test connectivity between subnets.

REQUIREMENTS:

cisco packet Tracer
Router
Switches
PCS
Ethernet cable.

PROCEDURE:

Creating Network Topology
open cisco packet tracer
create a new workspace
Add the required routers, switches and data,

IP addressing

LAN1	R1G010	192.168.1.1/27
LAN2	R1G01	192.168.2.1/27
LAN3	R2F0/0	192.168.3.1/27
LAN4	R2F01	192.168.4.1/27

classmate
Date
Page

For example;

IP address : 192.168.1.11
Subnet mask : 255.255.255.244
Default gateway : 192.168.1.1

Configuring Router R1

Open R1 → CH and enter:

enable
Configure terminal
IP address 192.168.1.1 255.255.244
no shutdown
Exit

Configure Router R2

enable
configure terminal
interface fast ethernet 0/1/0
IP add. 192.168.31.255.255.244
No shtdown
Exit

Configure PCs:

Open each PC → Desktop → IP configuration and enter appropriate IP address

PC	IP address	Subnet mask	gateway
PC1	192.168.1.11	255.255.244	192.168.1.1
PC2	192.168.1.12	255.255.244	192.168.1.1

Testing network.

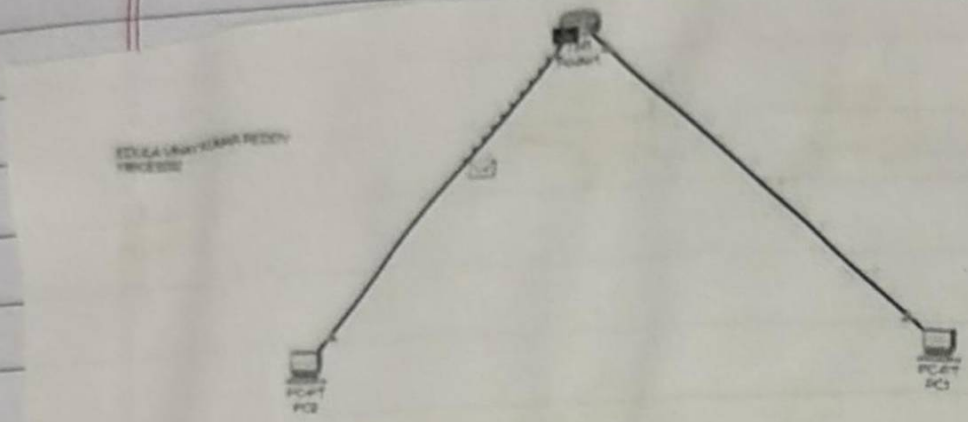
1. Open PC -> Desktop -> Command Prompt

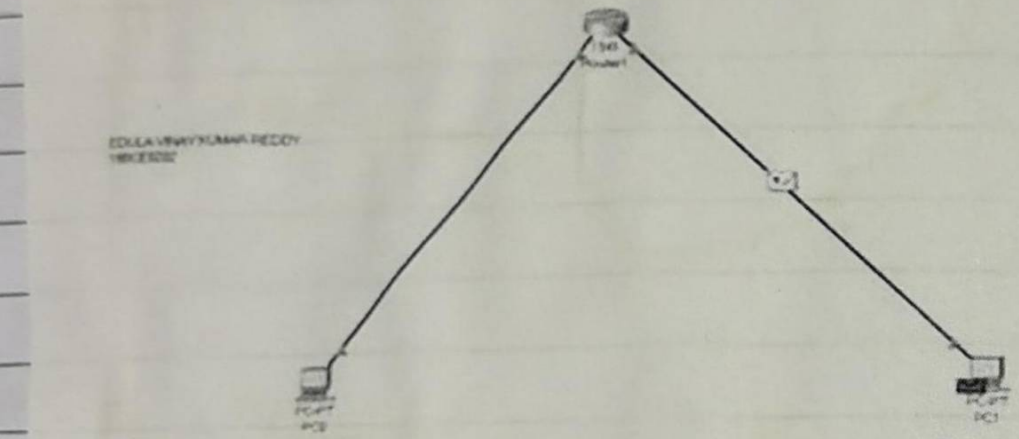
Ping 192.168.1.1

2) Test connectivity b/w PCs in subnet : 192.168.1.12

RESULT:

Thus the implementation of subnetting in Cisco Packet tracer was executed and verified successfully.





Ex:No 10 INTERNET WORKING
2418126 USING A ROUTER

AIM:

To design and configure a simple internetwork using a router and two in cisco packet tracer and test connectivity using PPC

PROCEDURE:

Step 1: Configure Router;

1) Open cisco packet tracer
2) Select Router → Routers,
3) Open CLI and press enter.
4 Configure Fast Ethernet
Interface Fast ethernet
No shutdown

Step 2: Configure PC0

Go to PC0 desktop → IP config
IP address: 192.168.10.2
Sub net mask: 255.255.255.0
Default Gateway: 192.168.10.

Step 3: Configure PC1

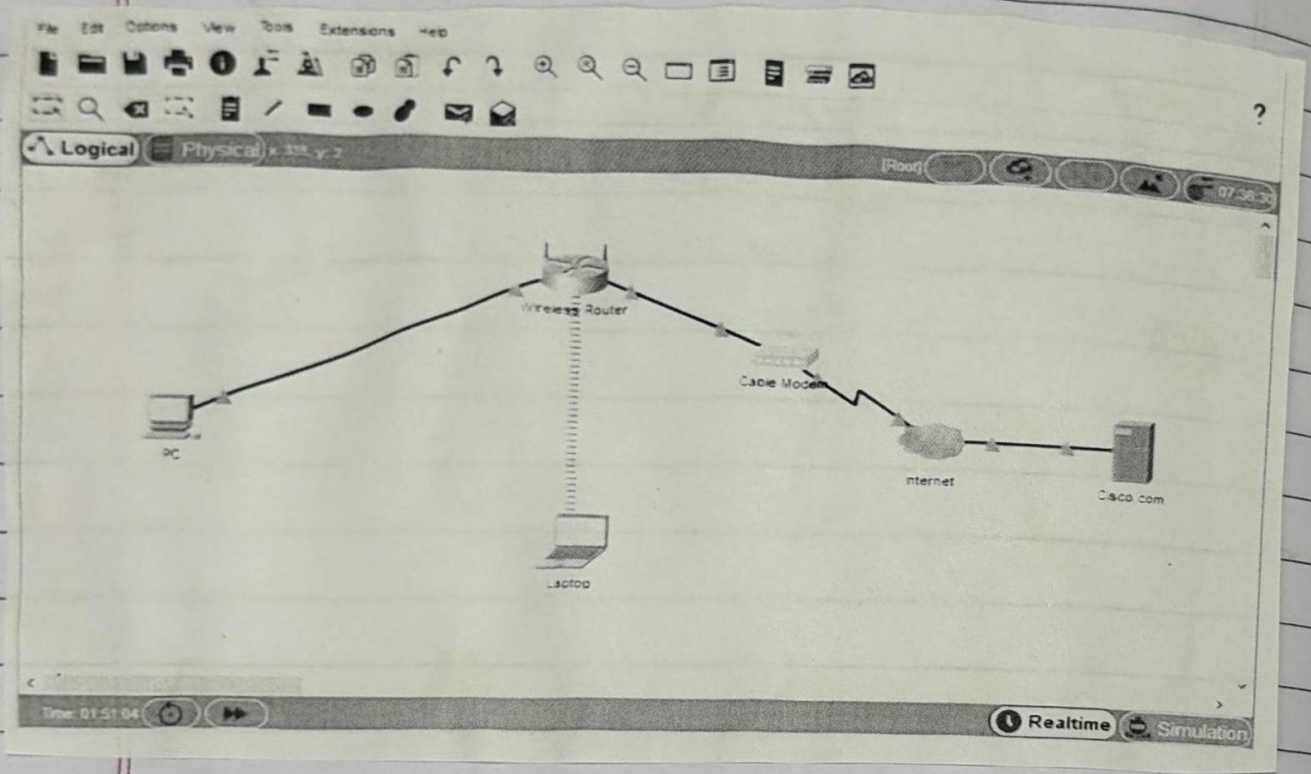
Go to PC1 → desktop →
IP config:
IP address: 192.168.20.2

Step 4: Connect the devices
use copper straight
PC0 Fast Ethernet
Ethernet 0,0
PC1 Fast Ethernet0.

Step 5: Test connectivity
1) Select simulation mode
2) Choose Add simple PDC
3) Click PC0 -> PC1
4) If PDU is successful they are communicating through the router.

RESULT:

The internet is successfully configured and connectivity is verified.


File Edit Options View Tools Extensions Help
Logical
Physical
[Root]
Wireless Router
Cable Modem
PC
Internet
Cisco.com
Laptop
Time: 01:51:04
Realtime
Simulation

classmate
Date
Page

EX: 10 C6)
30/8/26 WIRELESS ROUTER PATTERN, DHCP Server and internet cloud

AIM:
To design and configure and internetword using a wireless router DHCP sources and internet cloud.

PROCEDURE

Step 1: Build network
Add:
PC
wireless router
laptop
Cable Modem
Internet cloud
cisco.com server

step 2:
connect devices
use the appropriate cable
PC → wireless route → cable modem.
copper straight - through
wireless Router → cable Modem
copper straight - through
cable Modem → Internet cloud

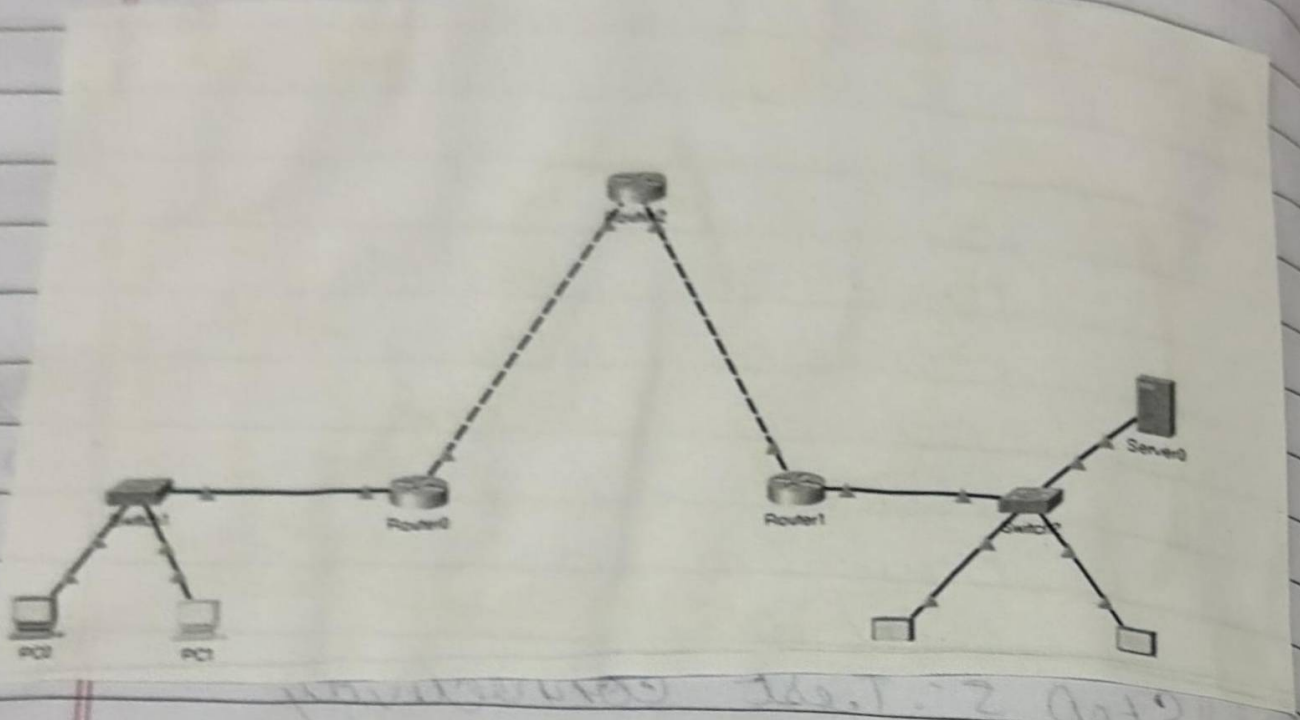
Step 3: configure wireless router
1) Open wireless Router → GUI
2) Go to wireless
3) Set DNS server
4) Click save setting

Step 4:
DHCP: ON
Pool Name: DHCP pool
default Gateway: 208.67.220.
Starting IP: 208.67.220.1
Maximum users: 50
Click add

Step 5: Test connectivity
1) Open PC → desktop → command prompt.
2) Type: ipconfig
3) Then ipconfig renew
4) Test server.

RESULT:

The wireless internet work was successfully designed using a wireless DHCP.


Router0
Router1
Server0
PC0
PC1

EX: 11 ROUTING AT NETWORK LAYER

30/8/2026

AIM:

To simulate and config. Static routing b/w multiple networks using cisco packet tracer, configure primary and backup static routes using administrative distance.

Requirements:

Cisco packet Tracer
Three routers Router 0, Router 1, 2
PCS/end devices.
Properly assigned IP addresses

Observation:

A network topology consisting of 3 routers, 1, 2, was created in cisco packet tracer, different network were connected and assigned IP address.

Router 1:

was configured with static routes to 10.0.0.0/8 to 40.0.0.0/8 networks.

The main link b/w router is disconnected to simulate a link failure

verification : Show ip router static. The router showed configured static router administrative using the pin command.

Deleting static route:

Show ip route static, Main route, the backup.

RESULT:

Thus, Static routing was configured and simulated using cisco.

Exp 12

Part A: Echo Client-Server using TCP Sockets

AIM

To write and implement an Echo Client-Server program using TCP sockets in C.

Algorithm

Server Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Put the server socket in listening mode using listen().
4. Accept an incoming client connection using accept().
5. Read the message sent by the client using read() or recv().
6. Echo the same message back to the client using write() or send().
7. Close the client and server sockets using close().

Client Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Connect to the server's IP address and port number using connect().
3. Prompt the user for input and send the message to the server using write() or send().

classmate
Date
Page

X: 12
22/9/26

IMPLEMENT ECHO CLIENT SERVER USING TCP/UDP SOCKETS

AIM:

To create and implement an echo client server program using TCP socket in C

RESULT:

Thus the echo client ser was successfully implem using sockets.

classmate
Date

Receive the echoed message back from the server using read() or recv().

5. Display the received message on the terminal.
6. Close the socket using close().

Code Implementation (TCP)

tcp_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 1. Create socket
```

```
xit(EXIT_FAILURE);
}

    // 2. Bind socket
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // 3. Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
}

printf("[TCP Server] Waiting for incoming connections on port
%d...\n", PORT);
```

```
    new_socket = accept(server_fd, (struct sockaddr
address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }

    // 5 & 6. Read and Echo back
    read(new_socket, buffer, BUFFER_SIZE);
    printf("[TCP Server] Received from client: %s\n", buffer);

    send(new_socket, buffer, strlen(buffer), 0);
    printf("[TCP Server] Echoed message back to client.\n");

    // 7. Close sockets
    close(new_socket);
    close(server_fd);
    return 0;
}
```

tcp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];

    // 1. Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <
0) {
```

Input/Output (TCP)

Input (Client Terminal):

Enter message to send to server: Hello Networks Lab!

Output (Server Terminal):

[TCP Server] Waiting for incoming connections on port 8080...

[TCP Server] Received from client: Hello Networks Lab!

[TCP Server] Echoed message back to client.

Output (Client Terminal):

[TCP Client] Message sent.

[TCP Client] Echo received from server: Hello Networks Lab!

Part B: Echo Client-Server using UDP Sockets

AIM

To write and implement an Echo Client-Server program using UDP sockets in C.

Algorithm

Server Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Receive a message and store the client's address using

```
// 3. Receive message
int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&cliaddr, &len);
    buffer[n] = '\0';
    printf("[UDP Server] Received from client: %s\n", buffer);

    // 4. Echo back message
    sendto(sockfd, (const char *)buffer, strlen(buffer),
MSG_CONFIRM, (const struct sockaddr *)&cliaddr, len);
    printf("[UDP Server] Echoed message back to client.\n");

    // 5. Close socket
    close(sockfd);
    return 0;
}
```

udp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
```

DP Client] Message sent.

[UDP Client] Echo received from server: Testing UDP Socke

classmate
Date
Page

EX: 12
22/9/26

IMPLEMENT ECHO CLIENT SERVER USING TCP/UDP SOCKETS

AIM:

To create and implement an echo client server program using TCP socket in C

EX: 13
26/9/26

PING PROGRAM

AIM:

To implement a Ping Program using Python raw Sockets to check the reachability of a host and calculate round trip time.

RESULT:

Thus ping Program using python to check the reachability of a host is calculated successfully

AIM

To implement a custom Ping program using raw ICMP (Internet Control Message Protocol) sockets in C.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Initialize a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_ICMP.
2. **Resolve Host IP:** Convert the domain name or IPv4 string into a valid target IP address using DNS lookup.
3. **Construct ICMP Packet:**
   - Set ICMP type to ICMP_ECHO (Type 8, Code 0).
   - Set Sequence Number and Process Identifier (PID) fields.
   - Calculate and set the Internet Checksum over the entire ICMP header and payload.
4. **Send Echo Request:** Transmit the packet to the destination address using sendto(). Record the transmission timestamp.
5. **Receive Echo Reply:**
   - Wait for incoming packets with recvfrom() within a timeout window.
   - Parse the outer IP header to extract the inner ICMP packet.

erify that the received packet is an ICMP_ECHOREPLY (Type 0) matching the process ID and sequence number.

6. **Calculate Metrics:** Subtract transmission timestamp from receive timestamp to obtain Round Trip Time (RTT).
7. **Display Output:** Print sequence number, TTL (Time to Live), packet size, and RTT. Repeat for desired number of pings.

Code Implementation (ping.c)

```
1  #include <stdio.h>
   #include <stdlib.h>
2
   #include <string.h>
   #include <unistd.h>
3
   #include <sys/types.h>
4.
   #include <sys/socket.h>
5.
   #include <netinet/in.h>
   #include <netinet/ip.h>
6.
   #include <netinet/ip_icmp.h>
   #include <arpa/inet.h>
7.
   #include <netdb.h>
   #include <sys/time.h>
1.

2.
   #define PACKET_SIZE 64
   #define PORT_NO 0
3.
   #define TIMEOUT 1 // Timeout in seconds
```

```
ard Internet Checksum Algorithm
ned short calculate_checksum(unsigned short *buffer, int
gth) {
    unsigned long sum = 0;
    while (length > 1) {
        sum += *buffer++;
        length -= 2;
    }
    if (length == 1) {
        sum += *(unsigned char *)buffer;
    }
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: sudo %s <hostname/IP>\n", argv[0]);
        return 1;
    }

    int sockfd;
```

```
kaddr_in dest_addr;
hostent *host;
r packet[PACKET_SIZE];
struct icmp *icmp_hdr;
struct timeval timeout, tv_start, tv_end;

// 1. Resolve Hostname
host = gethostbyname(argv[1]);
if (host == NULL) {
    perror("Could not resolve hostname");
    return 1;
}

memset(&dest_addr, 0, sizeof(dest_addr));
dest_addr.sin_family = AF_INET;
dest_addr.sin_addr = *((struct in_addr *)host->h_addr_list[0]);

// 2. Create Raw Socket for ICMP
sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sockfd < 0) {
    perror("Socket creation failed (run as root)");
    return 1;
}
```

```
struct ip *ip_hdr = (struct ip *)recv_buf;
int ip_hdr_len = ip_hdr->ip_hl * 4;
struct icmp *reply_icmp = (struct icmp *)(recv_buf +
ip_hdr_len);

// Calculate RTT in milliseconds
double rtt = (tv_end.tv_sec - tv_start.tv_sec) * 1000.0 +
             (tv_end.tv_usec - tv_start.tv_usec) / 1000.0;
```

Algo

Serv

1.

```
if (reply_icmp->icmp_type == ICMP_ECHOREPLY) {
```

2.

```
    printf("%d bytes from %s: icmp_seq=%d ttl=%d
time=%.2f ms\n",
```

3.

```
           bytes_rcvd - ip_hdr_len,
```

4.

```
           inet_ntoa(from_addr.sin_addr),
```

5.

```
           reply_icmp->icmp_seq,
```

6.

```
           ip_hdr->ip_ttl,
           rtt);
```

7.

```
    }
}
```

Clie

1.

```
sleep(1); // Wait 1 second between requests
}
```

2.

```
close(sockfd);
```

3. Pr

se

Server

1. Cr
   SC
2. Bir
   bir
3. Pu
4. Ac
5. Re
   rec
6. Ecl
   sen
7. Clo

Client S

1. Cre
   SO
2. Cor
   cor
3. Pro
   serve

...mand Terminal:

...do ./myping 127.0.0.1

Output

Server Terminal:

```
PING 127.0.0.1 (127.0.0.1) 64 bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.08 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.06 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.07 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.05 ms
```

EX: 14
30/9/26

NETWORT PACKET SUFFER USING ROW SOCKET

AIM:

To write and implement a network packet sniffer using row socket in to capture and parse IP, TCP and UIP.

RESULT:

Thus the network sniffer using row socket is implemented suddessfully to capture and parse IP, TCP, and UDP

EXP 14

AIM

To write and implement a network packet sniffer using Raw Sockets in C to capture and parse IP, TCP, and UDP network headers.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Open a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_TCP (or ETH_P_ALL for lower-level link layer capture).
2. **Buffer Allocation:** Allocate a buffer (e.g., 65536 bytes) to hold incoming network frames and packets.
3. **Capture Loop:**
    - Call recvfrom() to continuously capture network packets passing through the interface.
4. **Parse Headers:**
    - **Extract IP Header:** Cast the buffer pointer to an struct iphdr pointer. Extract fields like Version, Header Length, TTL, Protocol, Source IP, and Destination IP.
    - **Identify Protocol:** Check the protocol field (ip_hdr->protocol).
    - **Extract Transport Header:**
        - If Protocol is 6 (TCP): Cast buffer offset to struct tcphdr and extract Source Port, Destination Port, and Flags (SYN, ACK, FIN, etc.).

      If Protocol is 17 (UDP): Cast buffer offset to struct udphdr and extract Source Port, Destination Port, and Length.
5. **Display Packet Info:** Print formatted packet metadata to the console.
6. **Clean Up:** Close the socket upon receiving a termination signal.

Code Implementation (packet_sniffer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 65536

void process_packet(unsigned char *buffer, int size) {
  // Extract IP Header
  struct iphdr *ip_hdr = (struct iphdr *)buffer;
  unsigned int ip_hdr_len = ip_hdr->ihl * 4;

  struct sockaddr_in source, dest;
  memset(&source, 0, sizeof(source));
  source.sin_addr.s_addr = ip_hdr->saddr;

  memset(&dest, 0, sizeof(dest));
  dest.sin_addr.s_addr = ip_hdr->daddr;

  printf("\n================ Packet Captured
================\n");
```

```
        printf("IP Header:\n");
        printf("   |-IP Version        : %d\n", (unsigned int)ip_hdr->version);
        printf("   |-Header Length     : %d Bytes\n", ip_hdr_len);
        printf("   |-Time To Live (TTL): %d\n", ip_hdr->ttl);
        printf("   |-Protocol          : %d\n", ip_hdr->protocol);
        printf("   |-Source IP         : %s\n", inet_ntoa(source.sin_addr));
        printf("   |-Destination IP    : %s\n", inet_ntoa(dest.sin_addr));

        // Handle TCP Packets
        if (ip_hdr->protocol == IPPROTO_TCP) {
            struct tcphdr *tcp_hdr = (struct tcphdr *)(buffer + ip_hdr_len);
            printf("TCP Header:\n");
            printf("   |-Source Port       : %u\n", ntohs(tcp_hdr->source));
            printf("   |-Destination Port  : %u\n", ntohs(tcp_hdr->dest));
            printf("   |-Sequence Number   : %u\n", ntohl(tcp_hdr->seq));
            printf("   |-Acknowledge No    : %u\n", ntohl(tcp_hdr->ack_seq));
            printf("   |-SYN Flag          : %d\n", tcp_hdr->syn);
            printf("   |-ACK Flag          : %d\n", tcp_hdr->ack);
            printf("   |-FIN Flag          : %d\n", tcp_hdr->fin);
        }
        // Handle UDP Packets
        else if (ip_hdr->protocol == IPPROTO_UDP) {
            struct udphdr *udp_hdr = (struct udphdr *)(buffer + ip_hdr_len);
            printf("UDP Header:\n");
            printf("   |-Source Port       : %u\n", ntohs(udp_hdr->source));
            printf("   |-Destination Port  : %u\n", ntohs(udp_hdr->dest));
```

```
        printf("   |-UDP Length       : %u\n", ntohs(udp_hdr->len));
    }

printf("=========================================================
=\n");
}

int main() {
    int raw_sock;
    socklen_t saddr_len;
    struct sockaddr saddr;
    unsigned char *buffer = (unsigned char *)malloc(BUFFER_SIZE);

    if (buffer == NULL) {
        printf("Buffer allocation failed.\n");
        return 1;
    }

    // Create raw socket to capture IP packets
    raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (raw_sock < 0) {
        perror("Socket creation error (ensure you run with sudo)");
        free(buffer);
        return 1;
    }

    printf("Starting Packet Sniffer...\nPress Ctrl+C to stop.\n");
```

```
    // Capture 3 packets as an example output
    int packet_count = 0;
    while (packet_count < 3) {
        saddr_len = sizeof(saddr);
        int data_size = recvfrom(raw_sock, buffer, BUFFER_SIZE, 0,
&saddr, &saddr_len);

        if (data_size < 0) {
            perror("Recvfrom error");
            free(buffer);
            close(raw_sock);
            return 1;
        }

        process_packet(buffer, data_size);
        packet_count++;
    }

    close(raw_sock);
    free(buffer);
    return 0;
}
```

Input / Output

Input

Command Terminal:

./sniffer

Output

Terminal Screen:

```
Starting Packet Sniffer...
Press Ctrl+C to stop.

================ Packet Captured ================
IP Header:
   |-IP Version        : 4
   |-Header Length     : 20 Bytes
   |-Time To Live (TTL): 64
   |-Protocol          : 6
   |-Source IP         : 192.168.1.5
   |-Destination IP    : 93.184.216.34
TCP Header:
   |-Source Port       : 49152
   |-Destination Port  : 80
   |-Sequence Number   : 10423011
   |-Acknowledge No    : 0
   |-SYN Flag          : 1
   |-ACK Flag          : 0
   |-FIN Flag          : 0
=================================================
```

Ex: 15 TYPES OF WEB LOGS
5/10/26 USING WEBALIZER LOG ANALYSIS

AIM:

To analyse different types of web logs using the webalizer log analysis tool

RESULT:

Thus different types of web logs using the webalizer log analysis is analysed successfully.

EXP 15

AIM

To analyze different types of web logs using the Webalizer log analysis tool and generate web traffic reports.

Procedure

Step 1: Run Webalizer

1. Download and extract the Webalizer executable (webalizer.exe) on Windows.
2. Place the executable in a dedicated working folder (e.g., C:\webalizer).

Step 2: Input Web Log File

1. Obtain or download a sample web server access log file (e.g., access.log from Apache, Nginx, or IIS).
2. Move the log file into the same directory as webalizer.exe or update the webalizer.conf configuration file to point to the input log location:

```
LogFile access.log
OutputDir ./output
```

Step 3: Run Webalizer

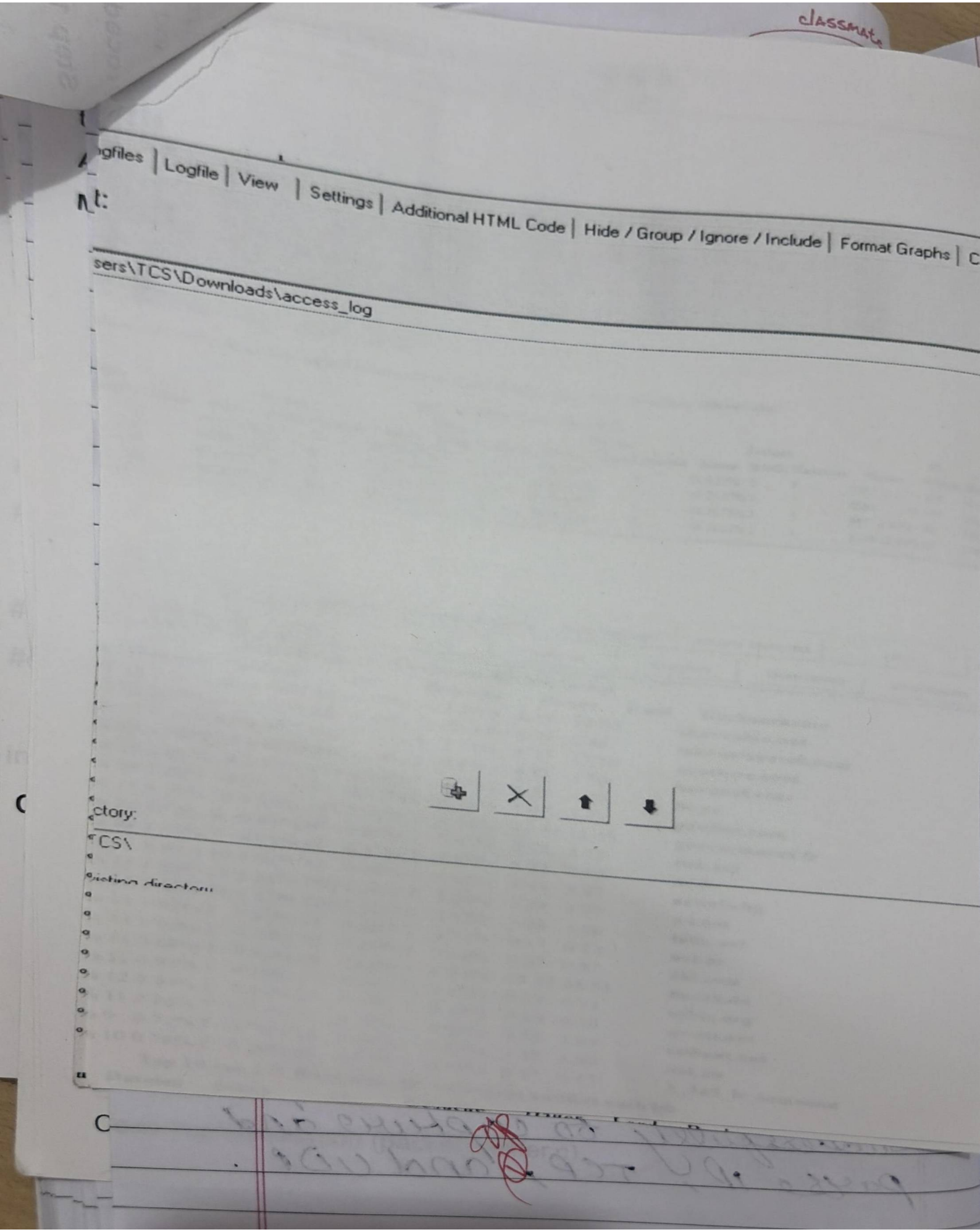
1. Open Command Prompt (cmd) and navigate to the Webalizer directory:

```
DOS
cd C:\webalizer
```

2. Execute Webalizer specifying the log file:

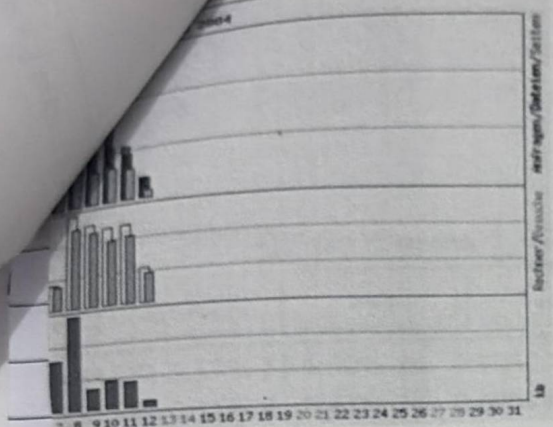
```
DOS
webalizer.exe access.log
```

3. Open the generated index.html file in any web browser to view the generated traffic report.


gfiles | Logfile | View | Settings | Additional HTML Code | Hide / Group / Ignore / Include | Format Graphs | C
sers\TCS\Downloads\access_log
ctory:
CS\

Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Hosts | Referrer | Searches | Usernames | Useragents | Countries



Abfragen (z.B. html, asp, txt); Dateien - erfolgreiche Anfragen für alle Typen (z.B. html, gif, css); Treffer - alle Anfragen, inklusive Fehler.

Tages-Statistik im Monat Maerz 2004

	Dateien			Seiten			Besuche			Rechner			kb		
ragen															
hnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Max
	29	147 11.54%	18	21	186 18.94%	23	27	15 6.41%	2	3	14 8.19%	2	3	2383 22.27%	298	542
	63	478 37.52%	21	56	419 42.67%	18	40	57 24.36%	2	7	48 28.07%	2	7	4509 42.14%	196	386
	29	205 16.09%	10	29	118 12.02%	6	16	49 20.94%	2	5	44 25.73%	2	5	955 8.93%	45	138
	102	205 16.09%	10	34	124 12.63%	6	28	46 19.66%	2	5	38 22.22%	2	4	1336 12.49%	67	379

Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents

Top 20 von 171 Rechnern (IP-Adressen)

n	Dateien		Seiten		kb		Besuche		Dauer		Land	Rechnername
%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62		inktomisearch.com
%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com
%	29	2.28%	14	1.43%	218	2.03%	7	2.99%	1.31	6.00		bc.ca
%	28	2.20%	14	1.43%	97	0.91%	1	0.43%	3.43	3.43		panduit.com
%	14	1.10%	10	1.02%	135	1.26%	1	0.43%	12.35	12.35		geovariances.fr
%	22	1.73%	8	0.81%	67	0.63%	1	0.43%	3.72	3.72		cox.net
%	19	1.49%	7	0.71%	61	0.57%	1	0.43%	1.52	1.52		ac.il
%	11	0.86%	10	1.02%	51	0.48%	2	0.85%	3.04	5.87		netinfo.bg
%	14	1.10%	13	1.32%	130	1.22%	3	1.28%	1.29	3.87		e-i.net
%	13	1.02%	13	1.32%	120	1.12%	1	0.43%	3.88	3.88		telia.net
%	13	1.02%	1	0.10%	28	0.26%	1	0.43%	0.13	0.13		net.ar
%	13	1.02%	2	0.20%	30	0.28%	1	0.43%	0.87	0.87		dhl.com
%	11	0.86%	9	0.92%	68	0.63%	1	0.43%	25.53	25.53		tiscali.de
%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.33	0.33		wtbts.org
%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.10	0.10		qwest.net
%	11	0.86%	7	0.71%	41	0.38%	2	0.85%	0.56	1.03		radiant.net
%	9	0.71%	7	0.71%	41	0.38%	1	0.43%	1.40	1.40		net.au
%	10	0.78%	2	0.20%	61	0.57%	1	0.43%	0.17	0.17		3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

n	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername

5.

6.

Co

Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents

.66% 71 5.57% 52 5.30% 86 0.80% 71 30.34% 0.09 6.62 **inktomisearch.com**

.26% 4 0.31% 4 0.41% 74 0.69% 4 1.71% 0.00 0.00 **rhyolite.com**

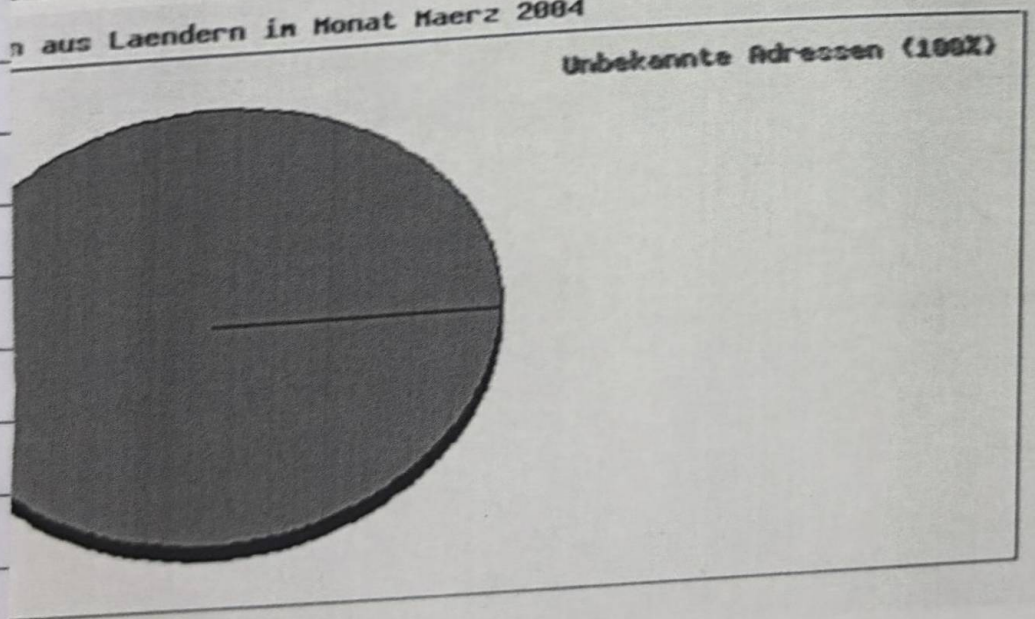
Top 1 von 1 Verweise

agen	Besuche	Verweise
0.00%	234 100.00%	- (Direkte Anfrage)

Top 1 von 1 Anwenderprogramme

gen	kb	Besuche	Anwenderprogramm
0.00%	10699 100.00%	234 100.00%	-

n aus Laendern in Monat Maerz 2004



Top 1 von 1 Ländern

en	Dateien	kb	Besuche	Land
81%	1271 99.76%	10684 99.85%	231 100.00%	Unbekannte Adressen

ro Soft free!